Team 38 Midterm Report
Beyond the Training Grid: Generalization and Uncertainty in Deep Energy Models for Delivery Drones

Aaryan Desai
MRSD, Robotics Institute
Carnegie Mellon University
aaryand@andrew.cmu.edu

Bill Ouyang
MRSD, Robotics Institute
Carnegie Mellon University
blouyang@andrew.cmu.edu

Ashutosh Kumar
MRSD, Robotics Institute
Carnegie Mellon University
ashutosk@andrew.cmu.edu

Yashasvi Sorapalli
MRSD, Robotics Institute
Carnegie Mellon University
ysorapal@andrew.cmu.edu

Abstract

Delivery operators must predict a drone's energy use before takeoff. Choudhry et al. [1] showed that a temporal convolutional network (TCN) trained on flight logs predicts quadrotor power better than a physics model, but tested it only on payload, speed and altitude levels seen in training. We ported their models to PyTorch, reproduced their results, and stress-tested them on held-out conditions against size-matched controls, scoring both only on flights neither saw. Holding out a payload, speed range or altitude raises the small TCN's power error by at most 0.4 percentage points (7–10% MAPE), and both TCNs beat the Liu physics model (11–14%) everywhere. On a new route and on a wind split we built from the raw anemometer and GNSS logs, both TCNs lose about one point. Retrained LSTMs are unreliable: 16 of 30 runs collapse to a constant, which we trace to dead ReLU units in the output head.

1 Introduction

A drone's power draw depends on its speed, climb rate, tilt, payload and the wind. Published energy models give substantially different estimates even for the same drone and conditions [5]. Choudhry et al. [1] instead learn power directly from flight data, but split flights at random, so every test flight reuses payload, speed and altitude levels seen in training.

Motivation. Operators and route planners need energy estimates they can trust before takeoff. A model that is accurate on familiar flights but wrong on new ones gives false confidence exactly when risk is highest.

Objectives. As proposed, we (i) measure how the published models degrade on held-out operating conditions, (ii) combine physics with a neural network, and (iii) add calibrated uncertainty (90% prediction intervals with ≈90% empirical coverage). Our results (Section 5) change (ii): the TCN barely degrades on held-out payload, speed and altitude, so the hybrid will target data efficiency and genuinely new conditions (route, wind) instead.

33rd Conference on Neural Information Processing Systems (NeurIPS 2019), Vancouver, Canada.

2 Related work and background

Choudhry et al. [1] predict power at each time step from a 20-step window of airspeed, its components, vertical speed and angle of attack, plus air density and payload. Their best TCN reaches 9.06% test MAPE versus 12.70% for the Liu physics model [4], and they score Monte Carlo energy distributions with Conditional Value-at-Risk (CVaR). We use their code [2], metrics and baselines, so our numbers are directly comparable. Liu et al. [4] model power as induced, profile and parasitic terms with five fitted constants; Zhang et al. [5] show that such physics models disagree widely. Rodrigues et al. [3] released the flight logs (DJI Matrice 100, Penn Hills, PA). For uncertainty we draw on deep ensembles [6] and Monte Carlo dropout [7].

3 Methods

3.1 Models

All networks map a window $x_{t-19:t} \in \mathbb{R}^{20\times 5}$ of time-varying inputs (airspeed x/y components, vertical speed, angle of attack, airspeed) and two static inputs (air density, payload) to the power $\hat{y}_t$ at the last step (Table 1). The TCN backbone is a stack of residual blocks of two causal dilated convolutions (kernel 2, dilations 1–16, ReLU), and its last-step features are concatenated with the static inputs. The LSTM backbone has three stacked layers and carries its state across a flight (stateful, batch size 1). Both feed a head of Linear→ReLU→Linear (10 hidden units).

Table 1: Models. Inputs: window $[B, 20, 5]$ + static $[B, 2]$; output: power $[B, 1]$. PyTorch LSTMs have two bias vectors per gate (Keras: one), hence 768 / 192 more parameters than in [1].

Model	Backbone (per-step feature map)	Head input	Parameters
Liu physics model [4]	closed form, 5 fitted constants	–	5
s-TCN	5 residual blocks, 16 filters: $[B, 20, 16]$	16 + 2	5,225
b-TCN	5 residual blocks, 64 filters: $[B, 20, 64]$	64 + 2	76,073
s-LSTM	3 layers, 16 units: $[B, 20, 16]$	16 + 2	6,025
b-LSTM	3 layers, 64 units: $[B, 20, 64]$	64 + 2	85,417

3.2 Dataset and splits

We use the authors' preprocessed flights [2]: 187 flights of a DJI Matrice 100 at 5 Hz, 179 on the main triangular route covering every combination of payload (0/250/500 g), speed (4–12 m/s) and altitude (25–100 m), and 8 on other routes.[1] The target is battery voltage × current. Inputs are min–max normalized with statistics from the training and validation flights (payload and density are left raw), and training samples batches of 32 overlapping 20-step windows from training flights only (TCN; the stateful LSTMs use batch size 1).

Besides the paper's random split (114/38/35 train/val/test flights), each held-out split tests on every main-route flight with one condition: 500 g (`payload500`, extrapolation), 250 g (`payload250`, interpolation), 12, 10–12 or 8–12 m/s (`speed12`, `speed10-12`, `speed8-12`), 100 m (`alt100`) or cruise wind above 4.34 m/s (`wind`); `route` tests on all off-route flights. Validation (25%, seed 42) is drawn from the remaining flights, so model selection never sees the held-out condition. Training sizes are in Table 3.

Wind split. The preprocessed data has no wind, so we estimate it from the raw logs [3]. In cruise, the wind is the ground velocity minus the airflow measured by the anemometer:

$$\mathbf{w} = \mathbf{v}_g - s\,(\sin a, \cos a), \qquad a = \alpha + \delta, \tag{1}$$

where $\mathbf{v}_g$ is the body-frame velocity rotated by the INS heading into (east, north), s and α are the anemometer speed and the compass direction the air comes from, and $\delta = -9.4°$ is the 2019 magnetic declination at the site (WMM2015). These conventions make the wind consistent between

[1] Our proposal quoted 195 + 10 flights from the full dataset release; 187 of them have preprocessed data. We take route labels from the release's `parameters.csv` [3], which corrects two flights that the authors' flight sheet mislabels.

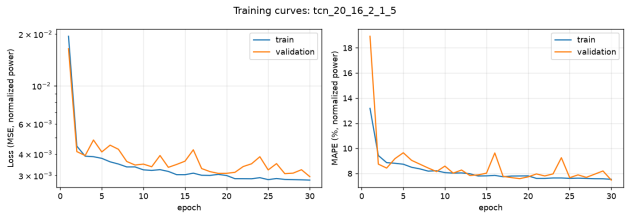


Figure 1: Retrained s-TCN (seed 0): training and validation loss and MAPE (on normalized power) per epoch; no overfitting.

the legs of the route; the alternative readings of the angle disagree 4–7× more. A flight's wind is its cruise mean, and `wind` holds out the windiest 20% of main-route flights.

3.3 Evaluation

Following [1], the power error of flight i with T_i scored steps, measured power y_t and prediction $\hat{y}_t$, and its energy error over the flight's regimes r (take-off, cruise, turns, landing, found by change-point detection on pitch) are

$$\mathrm{MAPE}_i = \frac{100\%}{T_i} \sum_{t=1}^{T_i} \left| \frac{y_t - \hat{y}_t}{y_t} \right|, \qquad \mathrm{RE}_i = \frac{100\%}{R_i} \sum_{r=1}^{R_i} \left| \frac{E_r - \hat{E}_r}{E_r} \right|, \tag{2}$$

where E_r is the Simpson-integrated energy of regime r. Both are averaged over flights, skipping each flight's first 19 steps (no full window). Two controls make the stress test fair. (1) **Size-matched control**: for each held-out split, a model trained on a random subset of the random split's training flights of the same size, which isolates the unseen condition from having less data. The random split trains on most windy flights, so the `wind` control instead tests on a random half of them and may train on the other half. (2) **Unseen flights only**: both models are scored on the n held-out flights that *neither* trained or validated on (Table 3). We report

$$\mathrm{gap} = \overline{\mathrm{MAPE}}_{\text{held-out}} - \overline{\mathrm{MAPE}}_{\text{control}}, \tag{3}$$

averaged over 3 seeds (mean ± std over seeds).

3.4 Loss and optimization

All networks minimize the mean squared error of normalized power. TCNs use Adam (learning rate 10^{-3}, $\epsilon = 10^{-7}$, batch size 32); LSTMs use RMSprop (learning rate 10^{-3}, $\rho = 0.9$, batch size 1).[2] We train for 30 epochs (the authors used 100; 30 already matches them for the TCNs and the s-LSTM, Table 2) and evaluate the final epoch.

4 Experiments

4.1 Baseline reproduction

The authors' Keras TCN and LSTM [2] code was ported to PyTorch and converted their checkpoints (output difference $< 4 \cdot 10^{-7}$). The converted models reproduce the paper's power MAPE to within 0.01 points (Table 2). The energy error matches for Liu and the LSTMs and is 0.2–0.4 points higher for the TCNs, as it also is with the authors' own TensorFlow code, so we attribute it to library versions rather than the port. Retrained from scratch, both TCNs match the paper over 3 seeds (s-TCN 8.86±0.26% vs 9.13%, b-TCN 8.92±0.16% vs 9.06% test MAPE), as does the s-LSTM (12.42% vs 12.36%, one seed). The b-LSTM does not (16.80% vs 11.47%; Section 6). Fig. 1 shows training curves.

[2] Our proposal stated a learning rate of 0.002. The released code passes the optimizer by name, so Keras uses its default of 10^{-3}; our port matches the released code.

Table 2: Reproduction: power MAPE / energy RE (%) on the paper's flight sets ("Random": 10 fixed flights, mostly off-route; "Test": 37-entry list). Top: authors' checkpoints converted to PyTorch; bottom: retrained from scratch.

	Paper [1]		Ours (PyTorch)	
Model	Random	Test	Random	Test
Liu	14.05 / 9.16	12.70 / 7.41	14.05 / 9.10	12.70 / 7.36
b-LSTM	13.58 / 10.35	11.47 / 7.57	13.58 / 10.37	11.47 / 7.59
s-LSTM	15.35 / 11.65	12.36 / 8.24	15.34 / 11.61	12.36 / 8.23
b-TCN	10.38 / 7.38	9.06 / 5.12	10.38 / 7.82	9.06 / 5.37
s-TCN	10.36 / 7.56	9.13 / 5.52	10.36 / 7.99	9.13 / 5.75
Retrained from scratch, 30 epochs				
b-TCN (3 seeds)	–	–	9.82±0.43 / 6.74±0.68	8.92±0.16 / 4.98±0.19
s-TCN (3 seeds)	–	–	9.74±0.59 / 6.38±0.71	8.86±0.26 / 4.85±0.28
b-LSTM (seed 0)	–	–	16.29 / 11.97	16.80 / 12.97
s-LSTM (seed 0)	–	–	12.66 / 7.32	12.42 / 7.44

Table 3: Stress test (3 seeds). Train: training flights; n: unseen held-out flights. Gap: held-out minus control (percentage points, positive = held-out worse). Liu: MAPE on the same flights.

Split	Train	n	s-TCN MAPE held-out / control	s-TCN gap MAPE / RE	b-TCN gap MAPE / RE	Liu
`payload500`	87	21	7.46±0.27 / 7.26±0.12	+0.20 / +0.90	−0.01 / +0.77	11.04
`payload250`	90	16	7.31±0.18 / 7.21±0.08	+0.11 / +0.11	−0.20 / −0.25	10.79
`speed12`	106	9	8.87±0.30 / 8.57±0.04	+0.30 / +0.61	+0.22 / +0.71	13.49
`speed10-12`	78	27	9.89±0.25 / 9.96±0.75	−0.07 / −0.34	+0.12 / +0.03	13.68
`speed8-12`	52	57	8.82±0.11 / 8.52±0.36	+0.30 / −0.25	+0.90 / −0.03	12.85
`alt100`	100	16	8.22±0.15 / 7.82±0.05	+0.40 / +0.39	+0.22 / +0.34	10.69
`route`	134	8	10.89±0.32 / 9.81±0.28	+1.08 / +2.30	+1.30 / +2.71	13.67
`wind`	107	20	11.55±0.21 / 10.79±0.25	+0.77 / +1.39	+1.22 / +1.81	13.94

4.2 Stress test

Both TCNs ran every held-out split and its control with 3 seeds (78 runs); the LSTMs ran `wind` and its control with 3 seeds and the other splits with one (b-LSTM all eleven, s-LSTM seven). The nested speed splits were added after `speed12` showed no gap. Liu is scored on the same flights.

5 Results

Holding out a grid condition barely hurts the TCNs (Table 3, Fig. 2). For the s-TCN it costs at most 0.4 MAPE points, about the spread between seeds; only `alt100` is consistent across seeds, and extrapolating to 500 g costs barely more than interpolating to 250 g. The b-TCN (15× the parameters) stays within ±0.3 points except `speed8-12` (+0.90 MAPE, but −0.03 RE). Both TCNs beat Liu on every split, by 2.0–4.6 points. We report no LSTM gaps, because most retrained LSTMs collapsed (Section 6).

Speed is not an extrapolation in this dataset (failure analysis). We expected the largest gap on speed, since parasitic power grows with airspeed, yet even `speed8-12` (trained on 4–6 m/s) shows none for the s-TCN. Mean flight power is nearly flat across commanded speeds (521–535 W), because take-off, climb, hover and landing dominate, and the *measured* airspeed the network sees includes wind: 4–6 m/s training flights already reach 14.1 m/s, versus 18.1 m/s on test.

New route. `route` has the largest gap (+1.08 MAPE, +2.30 RE), but it is not a clean comparison: all off-route flights are in the random split's test set, so the control has never seen them either.

Wind is a real shift. Unlike `route`'s, the `wind` control has seen windy flights (12 in training), yet both TCNs are worse on unseen windy flights by +0.77±0.33 (s-TCN) and +1.22±0.20 (b-TCN) MAPE points (+1.4 / +1.8 RE), 2–6 times the seed spread. Caveat: wind clusters by date (half of the 36 windy flights are from two days), so it is partly a date split.

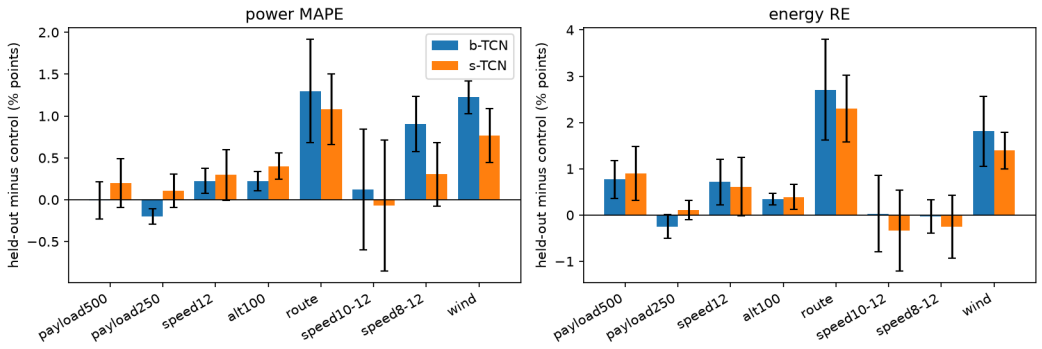


Figure 2: Held-out minus control per split for both TCNs, in power MAPE (left) and energy RE (right). Error bars combine the seed std of both models.

Ablation (training-set size). The controls share the random split's test flights, so they show what less data costs: test MAPE is 8.86±0.26 with 114 training flights, 8.64±0.13 with 87, 10.33±0.86 with 78 and 8.88±0.21 with 52. Halving the data costs nothing on average, but the 78-flight subset is the worst on every seed: *which* flights are used matters more than how many.

6 Discussion

Significance. The published TCN generalizes across the payload, speed and altitude grid far better than we expected; a planner can trust it on unseen grid points, but not yet on new routes or in wind. **Methodology matters.** Without size-matched controls and unseen-flight scoring, the "gap" would mix the condition shift, less training data and training error. **Challenges and limitations.** (i) Some comparisons rest on 8–21 flights, and the seed spread is as large as most gaps. (ii) The grid is coarse, and a held-out speed is still bracketed by measured airspeed. (iii) The wind estimate is noisy (≈4.5 m/s between legs), so it separates windy from calm days rather than ranking flights. (iv) The authors fit Liu's constants on all flights, which slightly favors Liu. (v) **Retrained LSTMs collapse.** 16 of our 30 LSTM runs predict a constant: all 10 ReLU units of the output head are inactive on every input, so the output is the last bias and no gradient reaches the LSTM. Survivors keep 1–4 active units, hence 3–6 point validation swings. We attribute this to batch-size-1 RMSprop (90k learning-rate-sized steps per epoch vs 2.8k averaged Adam steps for the TCN); our port matches the Keras defaults and the authors' script saves only periodic snapshots, so we cannot tell whether their runs were affected. **Risk.** A hybrid may not beat a TCN that already generalizes.

7 Next steps

1. **Physics + TCN hybrid** $\hat{y}_t = P_{\text{Liu}}(x_t) + g_\theta(x_{t-19:t})$, with Liu's constants refit per split, evaluated as a learning curve (5–52 training flights, several subsets per size) and on `route`/`wind`.
2. **Uncertainty:** a Gaussian variance head (NLL loss), a deep ensemble (our 3 seeds) and MC dropout; 90% interval coverage on the held-out splits, then the authors' CVaR pipeline.
3. **LSTMs and completeness:** fix LSTM training (leaky ReLU head, lower learning rate or Adam), run both LSTMs on every split with 3 seeds, add a date-matched `wind` control; defer the proposed transformer.

Team contributions and code

Code: `https://github.com/wouyang26/idl_project_38`. **Bill Ouyang**: PyTorch port and checkpoint conversion, reproduction of the paper's results, b-TCN stress-test runs. **Yashasvi Sorapalli**: 30-epoch s-TCN baseline and figures, first held-out splits, b-LSTM stress-test runs. **Ashutosh Kumar**: s-LSTM experiments and stress-test runs, evaluation fixes. **Aaryan Desai**: data pipeline, Liu evaluation, size-matched controls and unseen-flight scoring, nested speed splits, stress-test pipeline, s-TCN runs, wind split and its runs, LSTM collapse diagnosis.

References

[1] A. Choudhry, B. Moon, J. Patrikar, C. Samaras, and S. Scherer. CVaR-based flight energy risk assessment for multirotor UAVs using a deep energy model. In *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 262–268, 2021.

[2] A. Choudhry, B. Moon, and J. Patrikar. cvar-energy-risk-deep-model (code and pretrained models). `https://github.com/castacks/cvar-energy-risk-deep-model`, 2021.

[3] T. A. Rodrigues, J. Patrikar, A. Choudhry, et al. In-flight positional and energy use data set of a DJI Matrice 100 quadcopter for small package delivery. *Scientific Data*, 8, 2021. Data: `https://doi.org/10.1184/R1/12683453`.

[4] Z. Liu, R. Sengupta, and A. Kurzhanskiy. A power consumption model for multi-rotor small unmanned aircraft systems. In *International Conference on Unmanned Aircraft Systems (ICUAS)*, pp. 310–315, 2017.

[5] J. Zhang, J. F. Campbell, D. C. Sweeney II, and A. C. Hupman. Energy consumption models for delivery drones: A comparison and assessment. *Transportation Research Part D*, 90:102668, 2021.

[6] B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems 30*, 2017.

[7] Y. Gal and Z. Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *International Conference on Machine Learning (ICML)*, 2016.